

Lattice HD Wallets: Post-Quantum BIP32 Hierarchical Deterministic Wallets from Lattice Assumptions

Conor Deegan¹, James Fitzwater¹, Kamil Doruk Gur², and David Nugent¹

¹ Project Eleven

{cd,jf,dn}@projecteleven.com

² Independent Researcher

dgur1@proton.me

Abstract. Hierarchical deterministic (HD) wallets, standardized as BIP32, allow users to manage a tree of cryptographic key pairs from a single master seed. A defining feature is non-hardened derivation: child public keys can be derived from a parent public key alone, enabling watch-only wallets where a server generates fresh receiving addresses while the signing key remains offline. Existing constructions rely on the algebraic structure of elliptic curve public keys, and recovering this functionality in the post-quantum setting is an open problem. We present two post-quantum HD wallet constructions. The first uses ML-DSA and supports hardened (private-key-dependent) derivation with proofs of unlinkability and unforgeability. The second, our primary contribution, uses Raccoon-G, a variant of the Raccoon signature scheme with Gaussian-distributed secrets. We modify Raccoon-G to publish full, unrounded public keys, preserving linearity. Since the sum of Gaussians is again Gaussian with predictable wider variance, derived keys remain statistically close to freshly generated ones, enabling non-hardened public key derivation. We prove unlinkability and unforgeability under standard lattice assumptions, and introduce a method for generating rerandomizable Raccoon-G key pairs from fixed randomness with provable existential unforgeability. Both constructions are implemented in Rust. To our knowledge, this is the first post-quantum HD wallet construction that recovers BIP32’s full public key derivation functionality with provable security under standard assumptions.

1 Introduction

The introduction of Bitcoin [Nak08] established the first widely deployed decentralized financial system built on a public ledger. A series of protocol extensions, formalized as Bitcoin Improvement Proposals (BIPs) and their equivalents in other systems, have since expanded blockchain-based systems from minimal transaction frameworks into practical financial infrastructure.

Among these extensions, BIP32, commonly known as *hierarchical deterministic (HD) wallets* [Wui13], is foundational to how blockchain systems are deployed

in practice. HD wallets allow users to deterministically derive an entire tree of key pairs from a single master seed, eliminating the need to independently generate and back up keys for each account. However, deterministic key derivation alone is not what distinguishes HD wallets. A defining feature is *non-hardened derivation*: the ability to derive child public keys from a parent public key without the private key ever being present. This enables watch-only wallets, where a server generates fresh receiving addresses on demand while the signing key remains in cold storage. It is this separation of key generation from signing authority that makes HD wallets practically deployable for payment processors, custodial services, and any system that must generate addresses at scale. BIP32 has also been a source of theoretical interest, with researchers proving it secure under various models [ADE⁺20, DEF⁺21, DFL19, ZXE⁺23, ER22, DZLZ23].

A sufficiently powerful quantum computer can efficiently solve the structured number-theoretic problems, integer factorization and the discrete logarithm problem, that underpin the digital signature and key agreement schemes used throughout blockchain systems [NB⁺07]. To address this, the National Institute of Standards and Technology (NIST) initiated a post-quantum cryptography standardization process [NIS25] to identify quantum-resistant replacements. While replacing the core cryptographic primitives in a blockchain system is conceptually straightforward, the ecosystem built around these systems depends on extended properties that do not translate directly into the post-quantum setting.

HD wallets are a clear example of this difficulty. Non-hardened derivation in classical schemes such as BIP32 relies on the linear algebraic structure of elliptic curve public keys: a public key is a curve point, and derivation amounts to adding a scalar multiple of the generator. The result is another valid public key, indistinguishable from one generated independently. Lattice-based signature schemes present two distinct obstacles to reproducing this functionality.

First, standardized schemes such as ML-DSA [oST25] apply a rounding step to the public key during key generation, dropping low-order bits to reduce key sizes. This rounding is a lossy, non-linear operation: given only the rounded public key, it is impossible to compute the correct rounded child public key, because rounding does not distribute over addition. This is not a parameter choice or a security concern, it is a structural barrier that makes non-hardened derivation infeasible for any scheme that rounds its public keys.

Second, even in schemes where the public key retains its full algebraic structure, each round of key derivation adds noise. In lattice-based schemes, a public key takes the form $\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e}$, where $\mathbf{e}$ is a noise vector essential for security. Deriving a child key adds fresh noise, and the accumulated noise changes the statistical profile of derived keys, making them potentially distinguishable from freshly generated ones. This breaks the *unlinkability* guarantee that non-hardened derivation is meant to provide: an observer should not be able to determine whether two public keys share a common parent.

As a result, every existing post-quantum HD wallet proposal either abandons non-hardened derivation entirely, requiring the private key for every child key

derivation, or attempts it with significant limitations: complex auxiliary proofs for unlinkability, or no formal security analysis.

In this work, we present the first post-quantum HD wallet construction that recovers BIP32’s public key derivation functionality with provable security under standard lattice assumptions. We build and analyze two constructions:

1. **ML-DSA HD Wallet (hardened derivation).** Our first construction uses the NIST-standardized ML-DSA [oST25] signature scheme. It provides deterministic seed-based key derivation with full security proofs, but, like all constructions based on signatures with rounded public keys, it is limited to hardened paths, requiring the private key for every child derivation.
2. **Raccoon-G HD Wallet (non-hardened derivation).** Our primary contribution uses Raccoon with Gaussian secrets, a variant introduced in the Threshold Raccoon paper [DKM⁺24]. We refer to this variant as **Raccoon-G** throughout this paper. **Raccoon-G** addresses both obstacles. It publishes the full, unrounded public key, preserving the linear structure needed for child key derivation. And its Gaussian-distributed secrets are stable under addition: the sum of two Gaussians is again Gaussian with a predictable, slightly wider distribution. This enables public key rerandomization, deriving child public keys from a parent public key alone, while keeping the child key’s distribution close enough to a fresh key’s distribution that no efficient adversary can distinguish them. We prove unlinkability and unforgeability under standard lattice assumptions (MLWE, MSIS), without requiring the complex auxiliary machinery needed by alternative approaches.

As part of building the **Raccoon-G** wallet, we demonstrate a method to generate rerandomizable **Raccoon-G** public and private keys from fixed randomness and prove that the resulting scheme satisfies existential unforgeability under honestly rerandomized keys. We provide public implementations of both constructions.

The paper is structured as follows. For the rest of this section, we examine previous analyses of BIP32 in both classical and post-quantum settings. In Section 2 we give background on wallets and underlying primitives, in Section 3 we introduce the post-quantum signature candidates, in Section 4 we present our ML-DSA based HD wallet construction and its security, in Section 5 we present our **Raccoon-G** based HD wallet construction with non-hardened key derivation and its security, in Section 6 we present implementation details, and in Section 7 we discuss implications.

1.1 Related Work

Formalization and analysis of BIP32. Das et al. [DFL19] provided the first formal treatment of deterministic wallets, defining their security properties. Although HD wallets are a form of deterministic wallet, their analysis does not capture the full functionality of BIP32, in particular, the independent public key derivation. Subsequent work by Das et al. [DEF⁺21] addressed this gap,

examining all aspects of BIP32 including non-hardened key derivation. Our security analysis of PQ-HD wallets partially follows this work.

Post-quantum wallets. Despite the maturity of classical HD wallet analysis, post-quantum HD wallet constructions remain limited [DEMS24, SD25, YL24]. Alkadri et al. [ADE⁺20] proposed the first post-quantum deterministic wallet with security in the quantum random oracle model, relying on a signature scheme with rerandomizable keys, an approach we also adopt. However, their instantiation uses qTESLA [ABB⁺20], whereas our constructions use ML-DSA and Raccoon-G.

Concurrent and independent work. Seck and Roux-Langlois [SRL25] recently proposed a PQ-HD wallet achieving public key derivation by leveraging the identification scheme underlying Dilithium. Their construction inherits the overhead of rejection sampling and requires complex auxiliary proofs to establish unlinkability. Separately, Quantus-Network’s QIP2 [Smi] adopts a standardized approach using ML-DSA for deterministic seed-based key generation, but explicitly omits public key derivation and provides no security analysis.

Our ML-DSA construction is comparable to QIP2 in scope but includes a formal security analysis. Our Raccoon-G construction goes further: it achieves BIP32’s public key derivation functionality natively through Raccoon-G’s linear homomorphic structure, without the auxiliary proofs required by Seck and Roux-Langlois. By targeting Raccoon-G, we establish unlinkability from standard lattice hardness assumptions, providing a cleaner and more direct security argument.

2 Background

This section formalizes the HD wallet abstraction and defines its two security properties: unlinkability and unforgeability. All cryptographic preliminaries, including digital signature syntax, lattice hardness assumptions, and the Gaussian properties used in our proofs, are deferred to Appendix A.

2.1 Hierarchical Deterministic Wallets

An HD wallet allows a user to derive an entire tree of key pairs from a single master seed, producing fresh private keys, public keys, and addresses on demand. For users, this design means that a single protected backup can restore an entire financial identity across various accounts. At the same time, poor key isolation or a single point of compromise can expose an entire tree of funds and transaction history.

The setup takes a *seed* and produces a master extended key pair $(xpriv_0, xpub_0)$. Each node i then has a chain code cc_i used for child derivation and its own key pair $(xpriv_i, xpub_i)$. There are two types of child nodes:

- A *non-hardened* node can be derived from a parent with or without knowledge of xpriv_i . In either case, the resulting child public key $\text{xpub}_{i'}$ must be the same, enabling watch-only wallets that derive public keys without the private key.
- A *hardened* node can only be derived using xpriv_i . Deriving $\text{xpub}_{i'}$ from xpub_i alone is undefined.

Following [DEF⁺21], an HD wallet can be modeled as a tree $\mathcal{T}$ of height h with nodes $\mathcal{N}$, edges $\mathcal{E}$, and root $(0, 0) \in \mathcal{N}$, defined as a tuple $\mathcal{T} = (h, (0, 0), \mathcal{N}, \mathcal{E})$. Each node is labeled (ℓ, t) , where ℓ is its depth and t its position relative to the root $(0, 0)$. A node can derive only its immediate children, so we rely on paths to specify a node's position in the tree.

A path of length ℓ that ends in position t is denoted by the sequence of indices $\text{path}_\ell^t = (m, i_1, \dots, t')$, where i' denotes a hardened index.

An HD wallet HDWal is then a tuple of algorithms (Setup , $\text{CKDer}_{\text{priv}}$, $\text{CKDer}_{\text{pub}}$, Sign , Verify) where:

- Setup takes public parameters 1^λ as input and outputs a master secret key xpriv_0 , the master public key xpub_0 and the first chaincode $\text{cc}_{0,0}$.
- $\text{CKDer}_{\text{priv}}$ takes a secret key $\text{xpriv}_{\ell,s}$, a chain code $\text{cc}_{\ell,s}$, a path path_ℓ^s , and a position index t and outputs a secret key $\text{xpriv}_{\ell+1,t}$, a chain code $\text{cc}_{\ell+1,t}$ and a path $\text{path}_{\ell+1}^t$ at level $\ell + 1$ and position t .
- $\text{CKDer}_{\text{pub}}$ takes a public key $\text{xpub}_{\ell,s}$, a chain code $\text{cc}_{\ell,s}$, a path path_ℓ^s , and a position index t and outputs a public key $\text{xpub}_{\ell+1,t}$, a chain code $\text{cc}_{\ell+1,t}$ and a path $\text{path}_{\ell+1}^t$ at level $\ell + 1$ and position t .
- Sign takes a secret signing key $\text{xpriv}_{\ell,s}$ and a message μ and outputs a signature Ψ .
- Verify takes a verification key $\text{xpub}_{\ell,s}$, a signature Ψ , a message μ , and outputs 1 if the signature is valid and 0 otherwise.
- RetrievePK takes a secret signing key $\text{xpriv}_{\ell,s}$ and outputs the corresponding public key $\text{xpub}_{\ell,s}$.

Remark 1. Depending on the underlying signature scheme, it is not guaranteed that $\text{CKDer}_{\text{pub}}$ exists for a lattice construction. We discuss this in detail when we introduce our two post-quantum wallets.

Correctness requires that any secret key derived via either hardened or non-hardened derivation matches the public key obtained via the corresponding public derivation, i.e.,

$$\text{RetrievePK}(\text{CKDer}_{\text{priv}}(\text{xpriv}_{\ell,s}, \text{cc}_{\ell,s}, \text{path}_\ell^s, t)) = \text{CKDer}_{\text{pub}}(\text{xpub}_{\ell,s}, \text{cc}_{\ell,s}, \text{path}_\ell^s, t),$$

and that signatures produced by xpriv verify under xpub .

For hardened derivations in our constructions, we combine secret and public key derivation into a single algorithm CKDer that outputs both the child secret key and public key:

- CKDer takes a secret key $\text{xpriv}_{\ell,s}$, a chain code $\text{cc}_{\ell,s}$, a path path_{ℓ}^s and a position index t and outputs a secret key $\text{xpriv}_{\ell+1,t}$, a public key $\text{xpub}_{\ell+1,t}$, a chain code $\text{cc}_{\ell+1,t}$ and a path $\text{path}_{\ell+1}^t$ at level $\ell + 1$ and position t .

The two security notions we consider are *unlinkability* and *unforgeability*. In essence, unlinkability requires that the public keys at non-corrupted nodes be indistinguishable from independently generated keys.

Definition 1 (Unlinkability). HDWal is said to be *unlinkable* if for any PPT adversary $\mathcal{A}$ the probability of the unlinkability game depicted in Figure 1 returning 1 is negligible in λ .

$\text{Exp}_{\text{HDWal}}^{\text{UNL}}(\mathcal{A})$	$\mathcal{O}^{pk}(\text{path}_{\ell}^s t)$
$\mathcal{K}, \mathcal{CC} := \emptyset, b \leftarrow \{0, 1\}$	if $\mathcal{K}[\text{path}_{\ell}^s t] \neq \perp$
$\text{xpub}_0, \text{xpriv}_0, \text{cc}_{0,0} \leftarrow \text{HDWal.Setup}(1^\lambda)$	$\text{xpriv}_{\ell+1,t}, \text{xpub}_{\ell+1,t} := \mathcal{K}[\text{path}_{\ell}^s t]$
$\text{path}_{\ell}^{s*}, t^* \leftarrow \mathcal{A}(\mathcal{O}^{pk}, \mathcal{O}^{sign}, \text{xpub}_0)$	else
if $\mathcal{K}[\text{path}_{\ell}^{s*} t^*] \neq \perp$	$\text{cc} := \mathcal{CC}[\text{path}_{\ell}^s]$
return 0	$\text{xpriv}_{\ell+1,t}, \text{xpub}_{\ell+1,t}, \text{cc}_{\ell+1,t}, \text{path}_{\ell+1}^t :=$
$\text{xpriv}, \text{xpub} := \mathcal{K}[\text{path}_{\ell}^{s*}], \text{cc} := \mathcal{CC}[\text{path}_{\ell}^{s*}]$	HDWal.CKDer($\text{xpriv}, \text{cc}, \text{path}_{\ell}^s t$)
$\text{xpriv}_{\ell+1,t}^0, \text{xpub}_{\ell+1,t}^0, \text{cc}_{\ell+1,t}^0, \text{path}_{\ell+1}^t :=$	$\mathcal{CC}[\text{path}_{\ell+1}^t] := \text{cc}_{\ell+1,t}$
HDWal.CKDer($\text{xpriv}, \text{cc}, \text{path}_{\ell}^{s*} t^*$)	$\mathcal{K}[\text{path}_{\ell+1}^t] := (\text{xpriv}_{\ell+1,t}, \text{xpub}_{\ell+1,t})$
$\mathcal{CC}[\text{path}_{\ell+1}^t] := \text{cc}_{\ell+1,t}^0$	return $\text{xpub}_{\ell+1,t}$
$\text{xpub}'_0, \text{xpriv}'_0, \text{cc}'_{0,0} \leftarrow \text{HDWal.Setup}(1^\lambda)$	$\mathcal{O}^{cc}(\text{path})$
$\text{xpriv}_{\ell+1,t}^1, \text{xpub}_{\ell+1,t}^1, \text{cc}_{\ell+1,t}^1, \text{path}_{\ell+1}^t :=$	return $\mathcal{CC}[\text{path}]$
HDWal.CKDer($\text{xpriv}'_0, \text{cc}'_{0,0}, \text{path}_{\ell}^{s*} t^*$)	$\mathcal{O}^{sign}(\mu, \text{path})$
$\mathcal{K}[\text{path}_{\ell+1}^t] := (\text{xpriv}_{\ell+1,t}^b, \text{xpub}_{\ell+1,t}^b)$	if $\mathcal{K}[\text{path}] = \perp$
$b^* \leftarrow \mathcal{A}(\mathcal{O}^{pk}, \mathcal{O}^{cc}, \mathcal{O}^{sign})$	return $\perp$
return $b^* = b$	$\text{xpriv}, \text{xpub} := \mathcal{K}[\text{path}]$
	return HDWal.Sign(xpriv, μ)

Fig. 1. Unlinkability experiment $\text{Exp}_{\text{HDWal}}^{\text{UNL}}$. $\mathcal{A}$ can access different oracles at different stages of the game.

Unforgeability, on the other hand, ensures that the HD wallet construction does not allow anyone without knowledge of the secret keys to forge signatures (and consequently transactions).

Definition 2 (Unforgeability). HDWal is said to be *unforgeable* if for any PPT adversary $\mathcal{A}$ the probability of the unforgeability game depicted in Figure 2 returning 1 is negligible in λ .

$\text{Exp}_{\text{HDWal}}^{\text{UNF}}(\mathcal{A})$	$\mathcal{O}^{pk}(\text{path}_\ell^s, t)$
$\mathcal{K}, \mathcal{CC}, \mathcal{M} := \emptyset$ $\text{xpub}_0, \text{xpriv}_0, \text{cc}_{0,0} \leftarrow \text{HDWal.Setup}(1^\lambda)$ $\mu^*, \Psi^*, \text{path}^* \leftarrow \mathcal{A}(\mathcal{O}^{pk}, \mathcal{O}^{sign}, \text{xpub}_0, \text{cc}_{0,0})$ if $\mathcal{K}[\text{path}^*] = \perp$ return 0 $\text{xpriv}, \text{xpub} := \mathcal{K}[\text{path}^*]$ if $\mu^* \in \mathcal{M}[\text{path}^*]$ return 0 return $\text{HDWal.Verify}(\text{xpub}, \Psi^*, \mu^*)$	if $\mathcal{K}[\text{path}_\ell^s t] \neq \perp$ $\text{xpriv}_{\ell+1,t}, \text{xpub}_{\ell+1,t} := \mathcal{K}[\text{path}_\ell^s t]$ else $\text{cc} := \mathcal{CC}[\text{path}_\ell^s]$ $\text{xpriv}_{\ell+1,t}, \text{xpub}_{\ell+1,t}, \text{cc}_{\ell+1,t}, \text{path}_{\ell+1}^t :=$ $\text{HDWal.CKDer}(\text{xpriv}, \text{cc}, \text{path}_\ell^s t)$ $\mathcal{CC}[\text{path}_{\ell+1}^t] := \text{cc}_{\ell+1,t}$ $\mathcal{K}[\text{path}_{\ell+1}^t] := (\text{xpriv}_{\ell+1,t}, \text{xpub}_{\ell+1,t})$ return $\text{xpub}_{\ell+1,t}$ $\mathcal{O}^{sign}(\mu, \text{path})$ if $\mathcal{K}[\text{path}] = \perp$ return $\perp$ $\text{xpriv}, \text{xpub} := \mathcal{K}[\text{path}]$ $\mathcal{M}[\text{path}] := \mathcal{M}[\text{path}] \cup \{\mu\}$ return $\text{HDWal.Sign}(\text{xpriv}, \mu)$

Fig. 2. Unforgeability experiment $\text{Exp}_{\text{HDWal}}^{\text{UNF}}$.

3 Post-Quantum Lattice HD Wallets

3.1 PQ Signature Candidates

Recovering non-hardened public key derivation in the post-quantum setting requires a signature scheme satisfying two properties. First, the public key must preserve a linear algebraic structure so that child public keys can be computed from a parent public key without the private key. Second, the secret key distribution must be stable under addition so that derived keys remain statistically close to freshly generated ones, ensuring unlinkability. We now evaluate three lattice-based signature schemes against these requirements.

ML-DSA serves as the NIST-standardized version of the Dilithium signature scheme. The scheme relies on the Module Learning with Errors (M-LWE) and Module Short Integer Solution (M-SIS) problems. It uses bounded uniform secrets and applies a rounding step to the public key during key generation, which reduces key sizes but destroys linear structure. ML-DSA therefore satisfies neither requirement: rounding prevents public key derivation entirely, and bounded uniform distributions are not stable under addition.

We then consider Raccoon [dPKPR24], a lattice-based signature scheme submitted to round 1 of *Additional Digital Signature Schemes for the NIST Post-Quantum Cryptography Standardization Process*, where it was not selected to advance to Round 2. Raccoon was designed primarily for masking friendliness for

side-channel resistance. It employs a Fiat-Shamir without aborts architecture, avoiding the nonlinear rejection sampling steps present in ML-DSA. Raccoon relies on the same M-LWE and M-SIS assumptions as ML-DSA. Crucially, the algebraic structure of Raccoon’s public key $\mathbf{t} = A\mathbf{s} + \mathbf{e}$ is linear prior to rounding. The base Raccoon submission does round $\mathbf{t}$ in key generation, but in our wallet construction we publish the full, unrounded $\mathbf{t}$ (see Section 5), satisfying the first requirement. However, the base Raccoon submission uses sums-of-uniform distributions for its secrets, chosen specifically because discrete Gaussians are not amenable to efficient masking. Sums-of-uniform distributions are not stable under addition in the required sense, so the second requirement is not met.

This motivates our use of **Raccoon-G**, a variant of Raccoon with Gaussian-distributed secrets introduced in the Threshold Raccoon paper [DKM⁺24]. **Raccoon-G** replaces the sums-of-uniform distributions used in base Raccoon with discrete Gaussians. The sum of independent Gaussians is again Gaussian with a predictable wider distribution (Lemma 2), which ensures that derived secret keys remain in the same distributional family as freshly generated ones. Combined with the unrounded public key, this satisfies both requirements and enables non-hardened public key derivation with provable unlinkability under standard lattice assumptions. The tradeoffs include: **Raccoon-G** inherits the significantly larger signature sizes of the Raccoon architecture (a consequence of removing rejection sampling), and switching from sums-of-uniforms in base Raccoon to discrete Gaussians in **Raccoon-G** makes constant-time implementation more challenging.

Table 1. Post-Quantum Signature Scheme Comparison

Feature	ML-DSA	Raccoon-G
Secret Distribution	Uniform	Gaussian
Public Key Structure	Rounded (lossy)	Full (linear)
Non-Hardened Derivation	Not possible	Supported via Gaussian stability
Signing Architecture	Rejection Sampling	Fiat-Shamir without aborts
NIST Status	Standardized (FIPS 204)	Not on NIST track
Hardware Implementation	Straightforward	Challenging (Gaussian sampling)
Primary Advantage	NIST compliance	Enables non-hardened derivation
Primary Limitation	No public key derivation	Larger signatures

We now present two post-quantum HD wallet constructions. In Section 4, we instantiate a hardened-only wallet using ML-DSA, the NIST-standardized signature scheme. Since ML-DSA does not natively support public key rerandomization, this construction only provides hardened key derivation. In Section 5, we present a **Raccoon-G** based wallet that additionally supports non-hardened public key derivation, enabling watch-only wallets and mirroring the full functionality of the classical BIP32 specification.

4 ML-DSA HD Wallet

4.1 Construction

We instantiate a post-quantum HD wallet using ML-DSA as the underlying signature scheme and HMAC modeled as a random oracle. Since ML-DSA does not define a public key rerandomization algorithm `RandPK`, this construction supports only hardened key derivation via `CKDer`. Figure 3 depicts the protocol, ML-DSA PQWal.

Setup(1^λ)	CKDer($\text{xpriv}_{\ell,s}, \text{cc}_{\ell,s}, \text{path}_{\ell}^s, t$)
$\text{sd}_{0,0} \leftarrow \{0, 1\}^\lambda$	$(\text{sd}_{\ell+1,t}, \text{cc}_{\ell+1,t}) := \text{HMAC}(\text{xpriv}_{\ell,s}, \text{cc}_{\ell,s}, t)$
$\text{cc}_{0,0} \leftarrow \{0, 1\}^\lambda$	$(\text{sk}_{\ell+1,t}, \text{pk}_{\ell+1,t}) := \text{SIG.DetKeyGen}(1^\lambda, \text{sd}_{\ell+1,t})$
$\text{sk}_0, \text{pk}_0 := \text{SIG.DetKeyGen}(1^\lambda, \text{sd}_{0,0})$	$\text{path}_{\ell+1}^t := \text{path}_{\ell}^s t$
$\text{xpriv}_0 = (\text{sk}_0, \text{sd}_{0,0}); \text{xpub}_0 = (\text{pk}_0)$	$\text{xpriv}_{\ell+1,t} = (\text{sk}_{\ell+1,t}, \text{sd}_{\ell+1,t}); \text{xpub}_{\ell+1,t} = (\text{pk}_{\ell+1,t})$
return $(\text{xpriv}_0, \text{xpub}_0, \text{cc}_{0,0})$	return $(\text{xpriv}_{\ell+1,t}, \text{xpub}_{\ell+1,t}, \text{cc}_{\ell+1,t}, \text{path}_{\ell+1}^t)$
Sign($\text{xpriv}_{\ell,s}, \mu$)	
return $\Psi = \text{SIG.Sign}(\text{xpriv}_{\ell,s}, \mu)$	
Verify($\text{pk}_{\ell,s}, \Psi, \mu$)	
return $\text{SIG.Verify}(\text{pk}_{\ell,s}, \Psi, \mu)$	

Fig. 3. ML-DSA PQWal Construction.

Correctness follows immediately from the correctness of the underlying building blocks. Every HMAC output used as the seed in `CKDer` is deterministic and therefore generates the same valid ML-DSA key pair via `DetKeyGen`. Since each hardened node uses a fresh key pair for signing, the derived keys produce valid signatures as long as ML-DSA is correct.

4.2 Unlinkability

We first show that ML-DSA PQWal is unlinkable.

Theorem 1. *Let HMAC be modeled as a random oracle. Let ML-DSA be unforgeable with an adversary having at most $\epsilon_{\text{ML-DSA}}$ advantage against the unforgeability game. Then ML-DSA PQWal is unlinkable. More formally, a PPT adversary making q_{pk}, q_{hmac} queries to $\mathcal{O}^{pk}$ and HMAC has an advantage:*

$$\leq \frac{q_{hmac}(q_{pk} + 1)}{2^\lambda} + \epsilon_{\text{ML-DSA}}$$

in winning the $\text{Exp}_{\text{HDWal}}^{\text{UNF}}$ game.

Proof. We follow a similar proof to the unlinkability proof in [DFL19]. To avoid a trivial loss, $\mathcal{A}$ must not have previously queried the path_ℓ^s, t to $\mathcal{O}^{pk}$. First, note that the random oracle assumption makes $\text{sd}_{0,0}$ and $\mathcal{CC}[\text{path}_\ell^s || t]$ (both before and after $\mathcal{A}$'s output) indistinguishable from uniform. Since CKDer generates a new ML-DSA key pair from a deterministic input, both keys and signatures produced by the keys are identically distributed. Therefore $\text{xpub}_{\ell+1}^0$ and $\text{xpub}_{\ell+1}^1$ share the same distribution as xpub_0 , unless $\mathcal{A}$ correctly queries HMAC to derive the input for DetKeyGen.

$\mathcal{A}$ can only query $\mathcal{O}^{cc}$ after declaring path_ℓ^s, t , which means $\mathcal{A}$ makes at most q_{pk} queries to receive chain codes. Together with the chain code from the challenge derivation (which is independent of b , since only the key is challenged), there are at most $q_{pk} + 1$ chain codes that $\mathcal{A}$ can guess correctly and query the HMAC on. Consequently, the probability that $\mathcal{A}$ finds the correct input to an HMAC output is at most $(q_{pk} + 1)/2^\lambda$. Since $\mathcal{A}$ makes at most q_{hmac} oracle queries, the advantage is at most $q_{hmac}(q_{pk} + 1)/2^\lambda$ by a union bound.

All that remains is to show that the adversary cannot find the secret keys corresponding to the challenged xpub . If $\mathcal{A}$ can retrieve the corresponding xpriv to determine b , it can be used as a subroutine to break the unforgeability of ML-DSA. An attacker $\mathcal{B}$ against the unforgeability of ML-DSA can invoke $\mathcal{A}$ to retrieve the signing key of the challenged verification key and forge a signature without querying the signing oracle. Hence the advantage of $\mathcal{A}$ must also be bounded by $\epsilon_{\text{ML-DSA}}$.

This concludes the proof. $\square$

4.3 Unforgeability

Next, we show that ML-DSA PQWal is unforgeable under the same assumptions.

Theorem 2. *Let ML-DSA be UF-CMA secure against a PPT adversary with at most $\epsilon_{\text{UF-CMA}}$ advantage. Let $|\rho|$ be the size of the domain of DetKeyGen. Then ML-DSA PQWal is unforgeable. More formally, for a PPT adversary making q queries to HMAC and q_{pk} public key queries, the advantage is*

$$\leq (q_{pk} + 1) \cdot \epsilon_{\text{UF-CMA}} + \frac{q^2}{|\rho|}$$

in winning $\text{Exp}_{\text{HDWal}}^{\text{UNL}}$ game.

Proof. We define an adversary $\mathcal{C}$ against the unforgeability of ML-DSA using $\mathcal{A}$ as a subroutine. If $\mathcal{A}$ breaks the unforgeability of ML-DSA PQWal, then $\mathcal{C}$ breaks the unforgeability of ML-DSA. We show this through a series of hybrids.

Hybrid₀: This is the base unforgeability game. A key pair is generated and the public key is given to $\mathcal{A}$ alongside access to the oracles. Queries to the random oracle are traced by maintaining a table based indexed by input. The advantage of $\mathcal{A}$ in **Hybrid₀** is its advantage in the unforgeability game.

Hybrid₁: In this hybrid, the challenger aborts if a collision occurs between $\text{xpub}, \text{xpriv}$, for two different path_ℓ^s . The rest remains identical to **Hybrid₀**.

Since the keys are derived using *deterministic* key generation, two keys can only collide if they share the same randomness ρ ; this can only happen if two HMAC outputs collide. Since $\mathcal{A}$ makes q queries to HMAC, we bound the probability by $q^2/|\rho|$.

Now, because we know no collisions exist between the keys, the forgery must target a unique key pair. Since each key pair at path_ℓ^s is a freshly generated ML-DSA key, a forgery in **Hybrid₁** also constitutes a forgery under path_ℓ^{s*} key pair for ML-DSA. We now define an adversary $\mathcal{B}$ against the unforgeability of ML-DSA. Upon receiving an ML-DSA public key, $\mathcal{B}$ embeds the challenge public key as either the master public key or one of the derived public keys. When $\mathcal{A}$ submits a forgery, $\mathcal{B}$ also submits the same forgery.

$\mathcal{B}$ can correctly guess which node $\mathcal{A}$ targets based on the visited path via the public key oracle. Therefore $\mathcal{B}$ succeeds with probability $1/(q_{\text{pk}} + 1)$ by choosing the correct node in which to embed the challenge key. This concludes the proof. $\square$

Note that the logic above does not allow $\mathcal{A}$ to derive public keys independently because the ML-DSA construction does not define that capability; however, for the Raccoon-G version, we assume $\mathcal{A}$ can derive public keys independently based on a seen xpub .

5 Raccoon-G HD Wallet

5.1 Construction

In this section, we instantiate post-quantum HD wallets using the Raccoon-G signature scheme and HMAC modeled as a random oracle. Figure 4 depicts the protocol, Raccoon-G PQWal.

Similar to its ML-DSA counterpart, Raccoon-G PQWal’s correctness follows directly from the underlying building blocks. The hardened derivation CKDer is the same as the ML-DSA variant. For $\text{CKDer}_{\text{pub}}$ to work, however, we require a RandPK algorithm for the underlying signature scheme. While ML-DSA does not define such an algorithm, we define one for Raccoon-G and use it to instantiate Raccoon-G PQWal with public key randomization.

Rerandomizable Raccoon-G Public Keys: We follow the approach in [ADE⁺20], which generates a new pair of small secrets to update the key pair. We retain the notation from Raccoon-G and illustrate the public key randomization algorithm in Figure 5. Note that we use the full $\mathbf{t}$ as the verification key, instead of dropping $\nu_{\mathbf{t}}$ bits³, but do not change the signature algorithm. This increases the size of public keys, which are often stored on-chain. We also define a deterministic sampling algorithm $\text{Sam} : \mathcal{S}_\rho \rightarrow \mathcal{D}_{\sigma_{\mathbf{t}}}^\ell \times \mathcal{D}_{\sigma_{\mathbf{t}}}^k$ that expands a pseudorandom seed into two Gaussian vectors of width $\sigma_{\mathbf{t}}$.

³ For correctness, we still perform this rounding as part of signature generation.

$\text{Setup}(1^\lambda)$	$\text{CKDer}(\text{xpriv}_{\ell,s}, \text{cc}_{\ell,s}, \text{path}_{\ell}^s, t)$
$\text{sd}_{0,0} \leftarrow \{0, 1\}^\lambda$	$(\text{sd}_{\ell+1,t}, \text{cc}_{\ell+1,t}) := \text{HMAC}(\text{xpriv}_{\ell,s}, \text{cc}_{\ell,s}, t)$
$\text{cc}_{0,0} \leftarrow \{0, 1\}^\lambda$	$(\text{sk}_{\ell+1,t}, \text{pk}_{\ell+1,t}) := \text{SIG.DetKeyGen}(1^\lambda, \text{sd}_{\ell+1,t})$
$\text{sk}_0, \text{pk}_0 := \text{SIG.DetKeyGen}(1^\lambda, \text{sd}_{0,0})$	$\text{path}_{\ell+1}^t := \text{path}_{\ell}^s t$
$\text{xpriv}_0 = (\text{sk}_0, \text{sd}_{0,0}); \text{xpub}_0 = (\text{pk}_0)$	$\text{xpriv}_{\ell+1,t} = (\text{sk}_{\ell+1,t}, \text{sd}_{\ell+1,t}); \text{xpub}_{\ell+1,t} = (\text{pk}_{\ell+1,t})$
return $(\text{xpriv}_0, \text{xpub}_0, \text{cc}_{0,0})$	return $(\text{xpriv}_{\ell+1,t}, \text{xpub}_{\ell+1,t}, \text{cc}_{\ell+1,t}, \text{path}_{\ell+1}^t)$
$\text{Sign}(\text{xpriv}_{\ell,s}, \mu)$	$\text{CKDer}_{\text{pub}}(\text{pk}_{\ell,s}, \text{cc}_{\ell,s}, \text{path}_{\ell}^s, t)$
return $\Psi = \text{SIG.Sign}(\text{xpriv}_{\ell,s}, \mu)$	$(\omega, \text{cc}_{\ell+1,t}) := \text{HMAC}(\text{pk}_{\ell,s}, \text{cc}_{\ell,s}, t)$
	$\text{pk}_{\ell+1,t} := \text{SIG.RandPK}(\text{pk}_{\ell,s}, \omega)$
$\text{Verify}(\text{pk}_{\ell,s}, \Psi, \mu)$	$\text{path}_{\ell+1}^t := \text{path}_{\ell}^s t$
return $\text{SIG.Verify}(\text{pk}_{\ell,s}, \Psi, \mu)$	return $\text{pk}_{\ell+1,t}, \text{cc}_{\ell+1,t}, \text{path}_{\ell+1}^t$
	$\text{CKDer}_{\text{priv}}(\text{xpriv}_{\ell,s}, \text{cc}_{\ell,s}, \text{path}_{\ell}^s, t)$
	$(\omega, \text{cc}_{\ell+1,t}) := \text{HMAC}(\text{pk}_{\ell,s}, \text{cc}_{\ell,s}, t)$
	$\text{sk}_{\ell+1,t} := \text{SIG.RandSK}(\text{sk}_{\ell,s}, \omega)$
	$\text{pk}_{\ell+1,t} := \text{SIG.RandPK}(\text{pk}_{\ell,s}, \omega)$
	$\text{path}_{\ell+1}^t := \text{path}_{\ell}^s t$
	$\text{xpriv}_{\ell+1,t} = (\text{sk}_{\ell+1,t}); \text{xpub}_{\ell+1,t} = (\text{pk}_{\ell+1,t})$
	return $(\text{xpriv}_{\ell+1,t}, \text{xpub}_{\ell+1,t}, \text{cc}_{\ell+1,t}, \text{path}_{\ell+1}^t)$

Fig. 4. Raccoon-G PQWal Construction. Note $\text{CKDer}_{\text{priv}}$ is the private-key counterpart to $\text{CKDer}_{\text{pub}}$, allowing the key holder to derive the child secret key matching a non-hardened public key derived via $\text{CKDer}_{\text{pub}}$.

Remark 2. Although the construction with Raccoon-G allows public key rerandomization, this increases the noise of the keys and changes the distribution of signatures. This affects the maximum depth at which the signature distribution can be argued to be indistinguishable, which is part of the unlinkability argument. One can instead consider a “hybrid” approach where non-hardened derivations are limited per public key generated with DetKeyGen .

5.2 Signature Correctness under Rerandomized Keys

We now show that Raccoon-G signatures remain correct under rerandomized keys. Correctness follows as long as the accumulated noise from key rerandomization, which grows proportionally to the square root of the number of derivations, remains within the original correctness bound of Raccoon-G. We formalize this as follows:

RandPK($\mathbf{A}, \mathbf{t}, \rho$)	RandSK($\mathbf{s}, \rho$)
$\mathbf{s}', \mathbf{e}' \leftarrow \mathcal{D}_{\mathbf{t}}^{\ell} \times \mathcal{D}_{\mathbf{t}}^k := \text{Sam}(\rho)$	$\mathbf{s}', - := \text{Sam}(\rho)$
$\mathbf{t}' := \mathbf{A}\mathbf{s}' + \mathbf{e}' + \mathbf{t}$	return $\mathbf{s} + \mathbf{s}'$
return $\mathbf{t}'$	

Fig. 5. RandPK Algorithm for Raccoon-G

Lemma 1. *Let Raccoon-G be correct. Then Raccoon-G with RandPK is correct for the signature bound $B_{2,\mathbf{z}} = e^{1/4} \cdot (\sqrt{n+1}\tau\sigma_{\mathbf{t}} + \sigma_{\mathbf{w}})\sqrt{N(k+\ell)} + (\tau \cdot 2^{\nu_{\mathbf{t}}} + 2^{(\nu_{\mathbf{w}}+1)}) \cdot \sqrt{Nk}$.*

Proof. The proof follows trivially from Appendix C.2 of [DKM⁺24]. After the t -th key derivation, by Lemma 2, $\mathbf{s}$ and $\mathbf{e}$ are statistically close to Gaussians with width $\sigma_{\mathbf{t}}\sqrt{t}$. However, the signing randomness width $\sigma_{\mathbf{w}}$ remains unchanged, as it is independent of the key derivation depth. The updated bound $B_{2,\mathbf{z}}$ accommodates this wider key noise, as we verify in Appendix C. This concludes the proof. $\square$

5.3 Signature Unforgeability under Rerandomized Keys

Before discussing the security of Raccoon-G PQWal, we show that Raccoon-G with RandPK is UF-CMA-HRK secure.

Theorem 3. *Let Raccoon-G be UF-CMA secure with the parameters given in Table 2. Then Raccoon-G with RandPK depicted in Figure 5 is UF-CMA-HRK secure.*

Proof. We define an adversary $\mathcal{B}$ that uses the adversary $\mathcal{A}$ against UF-CMA-HRK security as a subroutine. If $\mathcal{A}$ outputs a forgery $\mathbf{z}^*, \mathbf{h}^*, c^*, \rho^*$, we build $\mathcal{B}$ as follows: If $\rho = \{\}$, the signature uses the non-randomized public key $\mathbf{t}$; in this case, $\mathcal{B}$ can submit $\mathbf{z}^*, \mathbf{h}^*, c^*$ directly as a forgery. If $\rho \neq \{\}$, $\mathcal{B}$ can modify the given forgery to obtain a forgery for the base Raccoon-G. Upon receiving $\mathbf{z}^*, \mathbf{h}^*, c^*, \rho^*$, $\mathcal{B}$ runs Sam on ρ^* to obtain $\mathbf{s}', \mathbf{e}'$ and computes:

$$\begin{aligned}
\mathbf{z}' &:= \mathbf{z}^* - c^* \cdot \mathbf{s}' \\
\mathbf{t}' &:= \mathbf{t} - \mathbf{A}\mathbf{s}' - \mathbf{e}' \\
\mathbf{w}' &:= \mathbf{h}^* + \lfloor \mathbf{A}\mathbf{z}^* - 2^{\nu_{\mathbf{t}}} \cdot c^* \cdot \lfloor \mathbf{t} \rfloor_{\nu_{\mathbf{t}}} \rfloor_{\nu_{\mathbf{w}}} \\
\text{vk}' &= (\mathbf{A}, \mathbf{t}') \\
c' &:= \text{H}(\text{vk}', \mu, \mathbf{w}') \\
\mathbf{h}' &:= \mathbf{w}' - \lfloor \mathbf{A}\mathbf{z}' - 2^{\nu_{\mathbf{t}}} \cdot c' \cdot \lfloor \mathbf{t}' \rfloor_{\nu_{\mathbf{t}}} \rfloor_{\nu_{\mathbf{w}}}
\end{aligned}$$

and submits $\mathbf{z}', \mathbf{h}', c'$ as the forgery. Note that for $\mathbf{z}'$ to verify, $\mathbf{z}'$ also has to be bounded by $B_{2,\mathbf{z}}$. Since $\mathbf{z}^*$ is a valid forgery, it is a signature under a key

with at most n rerandomizations. Since $B_{2,\mathbf{z}}$ is computed to accommodate $n+1$ rerandomizations, $\mathbf{z}'$ is still bounded by $B_{2,\mathbf{z}}$.

Since $\mathbf{t}'$ corresponds to the base key obtained by removing the rerandomization terms $\mathbf{s}', \mathbf{e}'$, the linearity of the operations ensures that $\mathbf{t}'$ is correctly computed from $\mathbf{t}$. Since the original signature verifies for $\mathbf{t}, \mathbf{z}^*, \mathbf{h}^*, c^*$ will not verify under $\mathbf{t}'$. However, since $\mathbf{z}^*, \mathbf{h}^*, c^*$ is a valid signature under the original $\mathbf{A}, \mathbf{s}, \mathbf{e}$, $\mathbf{w}'$ can correctly be computed using $\mathbf{t}$. Hence, $\mathbf{h}', c'$ can be computed using $\mathbf{z}'$ and $\mathbf{w}'$. Hence $\mathbf{z}', \mathbf{h}', c'$ will verify under the base verification algorithm and therefore constitute a valid forgery against the base Raccoon-G. This concludes the proof. $\square$

Note that for key rerandomization to be secure, Raccoon-G needs to remain secure for parameters accommodating key rerandomization. For Hint-MLWE/MLWE, this is trivial, as we noted in the description of the problem. For Self-Target MSIS/MSIS, however, we need to make sure the problem is hard for the maximum number of rerandomizations n instead. We discuss how these affect the security proof in Appendix C and give the exact parameter constraints for correctness and security in Table 2.

5.4 Wallet Unforgeability

We now extend the wallet unforgeability result to Raccoon-G PQWal. The proof structure follows that of Theorem 2, with the main difference being that unforgeability now relies on the UF-CMA-HRK security of Raccoon-G to account for key rerandomization. We have the following corollary:

Corollary 1. *Let Raccoon-G be UF-CMA-HRK secure against a PPT adversary that has at most $\epsilon_{\text{UF-CMA-HRK}}$ advantage. Let $|\rho|$ be the size of the domain of DetKeyGen. Then Raccoon-G PQWal is unforgeable. More formally, a PPT adversary making q queries to HMAC and q_{pk} hardened public key derivation queries has advantage*

$$\leq (q_{\text{pk}} + 1) \cdot \epsilon_{\text{UF-CMA-HRK}} + \frac{q^2}{|\rho|}$$

in winning the $\text{Exp}_{\text{HDWal}}^{\text{UNF}}$ game.

Proof (Sketch). The proof is similar to the proof of Theorem 2 with the following modifications. As part of the unforgeability game, each entry in the chaincode table also contains an index that indicates where the parent node is. This way, the simulator can recursively derive non-hardened key paths even under changes to the initial chaincode and key randomness, as is done in Theorem 2. At each depth $t \leq n$, we need $\text{MLWE}_{k,\ell,\sigma_t\sqrt{t}}$ to be secure to ensure public keys are indistinguishable from uniform. By the trivial hierarchy noted as part of the MLWE definition, as long as $\text{MLWE}_{k,\ell,\sigma_t}$ holds, so will $\text{MLWE}_{k,\ell,\sigma_t\sqrt{t}}$ for $t \leq n$. Therefore, every derived public key is also indistinguishable from uniform, and the unforgeability argument remains the same.

Since the adversary can derive the non-hardened public keys on its own, the oracles are parameterized with an additional flag to indicate whether they are hardened or non-hardened. Oracles abort if a hardened verification key is queried on a non-hardened path or a signature is requested on a non-existent path. For non-hardened paths, the oracles can derive the relevant signing and verification keys by following the deterministic path and calling `RandSK` and `RandPK` accordingly.

The same logic regarding signature unforgeability follows from the ML-DSA case, but instead the challenger guesses which hardened node is targeted for forgery. The rest follows from the unforgeability under honest rerandomization, since each non-hardened path is a rerandomization of the same hardened node.

5.5 Wallet Unlinkability

We now prove that `Raccoon-G PQWal` is unlinkable. Unlike the ML-DSA case, the proof must additionally address the fact that signature distributions vary with the key derivation depth. The security reduction relies on two Hint-MLWE instances: one at the maximum rerandomization depth and one at depth 1.

Corollary 2. *Let `HMAC` be modeled as a random oracle, `H` be modeled as a programmable random oracle with challenge space $\mathcal{C}$. Let $\text{H-MLWE}_{k,\ell,\sigma_t\sqrt{n},\sigma_w,Q}$ and $\text{H-MLWE}_{k,\ell,\sigma_t,\sigma_w,Q}$ be secure, and let an adversary against these have $\epsilon_{\text{H-MLWE},n}$ and $\epsilon_{\text{H-MLWE},1}$ advantage in the Hint-MLWE games respectively. Then `Raccoon-G PQWal` is unlinkable. More formally, a PPT adversary making $q_{pk}, q_{hmac}, q_{\text{Sign}}, q_H$ public key, `HMAC`, signing, and `H` queries respectively has advantage*

$$\leq \frac{q_{hmac}(q_{pk} + 1)}{2^\lambda} + \frac{q_H(q_{\text{Sign}} + 1)}{|\mathcal{C}|} + \epsilon_{\text{H-MLWE},1} + \epsilon_{\text{H-MLWE},n}$$

Proof. Unlike the proof of Theorem 1, we also need to argue against a trivial attack where the adversary can request signatures at significantly different depths. Each key rerandomization adds noise to the signing key and therefore slightly changes the distribution of signatures. If the distribution of signatures at depth $\leq n$ is significantly different from those at depth 1, $\mathcal{A}$ has non-negligible advantage in distinguishing keys, which leaks non-negligible information about them. We first argue unlinkability without this attack, then show that $\mathcal{A}$'s view does not change depending on the depth of the signature.

The unlinkability proof proceeds as in Theorem 1. The only difference is that two different `xpub` can also be derived with `CKDerpub`. While `xpriv` have different distributions at different depths, by the trivial hierarchy between $\text{MLWE}_{k,\ell,\sigma_t}$ and $\text{MLWE}_{k,\ell,\sigma_t\sqrt{n}}$ samples, public keys at greater depths are still indistinguishable from random by the MLWE assumption. Therefore, every `xpub` is still identically distributed.

We will now argue that the adversary's view does not change if the depth- n signatures are replaced with depth-1 signatures. We will show this through a series of hybrids. Our observation comes from the main goal of the Hint-MLWE paper [KLSS23] where proofs of knowledge of specific structure can be simulated.

Hybrid₀: This is the base unlinkability game. Everything proceeds as before and the adversary is presented with depth n signatures if asked.

Hybrid₁: The challenger now changes how challenges are created for the signature. Instead of querying the oracle at $vk, \mu, \mathbf{w}$, the signing oracle first samples a random c in challenge space $\mathcal{C}$ and programs the oracle to give c at $vk, \mu, \mathbf{w}$ instead. If the programming fails, the challenger aborts. The rest is the same as **Hybrid₀**.

By the random oracle assumption, c in **Hybrid₀** and **Hybrid₁** are identically distributed. Then, **Hybrid₀** and **Hybrid₁** are indistinguishable as long as oracle programming does not fail. This can only happen if the signing oracle tries to program the challenge oracle at a point that $\mathcal{A}$ has already queried. By a union bound, this only happens with probability lower than $q_H(q_{\text{Sign}} + 1)/|\mathcal{C}|$.

Hybrid₂: Now that the challenges are not conditioned on $\mathbf{w}$, the signing oracle computes $\mathbf{w}$ using the public key instead. On a signing query, the oracle still samples $\mathbf{r}, \mathbf{e}'$ but computes $\mathbf{w}$ as $\lfloor \mathbf{A}\mathbf{z} - c \cdot \mathbf{t} + c \cdot \mathbf{e} + \mathbf{e}' \rfloor_{\nu_{\mathbf{w}}}$ instead. The rest is the same.

By simple arithmetic we have $\lfloor \mathbf{A}\mathbf{z} - c \cdot \mathbf{t} + c \cdot \mathbf{e} + \mathbf{e}' \rfloor_{\nu_{\mathbf{w}}} = \lfloor \mathbf{A}\mathbf{z} - c \cdot \mathbf{A}\mathbf{s} + \mathbf{e}' \rfloor_{\nu_{\mathbf{w}}} = \lfloor \mathbf{A}\mathbf{r} + \mathbf{e}' \rfloor_{\nu_{\mathbf{w}}}$. Since the oracle has access to both $\mathbf{e}$ and $\mathbf{e}'$, $\mathbf{w}$ can be computed without any external information. Thus, **Hybrid₁** and **Hybrid₂** are indistinguishable.

Hybrid₃: The challenger now changes how public keys at depth n are handled. Instead of giving the correct $\mathbf{t}$, the oracle gives a uniformly random vector $\mathbf{u}$ instead. Whenever a signature is issued for the key, the oracle uses $\mathbf{u}$ to program c . The rest is the same.

Since $\text{H-MLWE}_{k, \ell, \sigma_{\mathbf{t}} \sqrt{n}, \sigma_{\mathbf{w}}, Q}$ is secure, the challenger can embed the depth n verification key with a Hint-MLWE challenge. Then any advantage $\mathcal{A}$ has in distinguishing **Hybrid₃** from **Hybrid₂** would be a direct advantage for the challenger against the Hint-MLWE game, where the Hint-MLWE challenge is embedded in a standard way as $\mathbf{z}$ and $\mathbf{t}$ in the scheme, with $\mathbf{h}$ computed afterwards. Since Hint-MLWE is secure, the advantage of $\mathcal{A}$ is negligible.

Note that since $\mathbf{w}$ is conditioned on $\mathbf{t}$ in the previous hybrid, the signature bound must also be checked. However, since $\mathbf{h} = \mathbf{w} - \mathbf{y}$, the public key term cancels out and the signature verification still succeeds.

Hybrid₄: The challenger now changes how $\mathbf{z}$ in signatures are generated. The signing oracle samples a depth 1 signing key $\mathbf{s}, \mathbf{e}$ and for each depth n signature $c, \mathbf{z}, \mathbf{h}$, uses this signing key to compute $\mathbf{z}, \mathbf{w}$ instead. The signing continues as before.

Since the distribution of signature verification is conditioned on the random vector $\mathbf{u}$ in **Hybrid₄**, the change in how $\mathbf{z}$ is computed does not give a significant advantage to $\mathcal{A}$ unless verification bounds fail. Since a depth-1 signature is a Raccoon-G signature, we have:

$$B_{2, \mathbf{z}} = e^{1/4} \cdot (\tau \sigma_{\mathbf{t}} + \sigma_{\mathbf{w}}) \sqrt{N(k + \ell)} + (\tau \cdot 2^{\nu_{\mathbf{t}}} + 2^{(\nu_{\mathbf{w}} + 1)}) \cdot \sqrt{Nk}$$

which is smaller than the bound for a depth- n signature. The signature will then still verify. All we need is to argue that $\mathbf{z}$ does not leak significant information about the change in signing key over Q queries. For the Hint-MLWE argument to succeed, hints ($\mathbf{z}$ in this case) should not leak anything statistically meaningful regarding $\mathbf{s}$ in Q queries, sufficient to distinguish $\mathbf{A}\mathbf{s} + \mathbf{e}$ from uniform. This effectively makes $\mathbf{z}$ indistinguishable from a non-balanced Gaussian of width $\sigma_{\mathbf{w}}$, which dictates the bounds $\sigma_{\mathbf{w}} \geq 2\sqrt{B_{\text{H-MLWE}} \cdot (k + \ell) \cdot \log N}$ and $\sigma_{\mathbf{w}} \geq \tau \cdot \sigma_{\mathbf{t}}$. For our choice of parameters, this is the case for both versions of $\mathbf{s}$ at depth 1 and depth n , meaning $\mathbf{z}$ behaves like a non-balanced Gaussian of width $\sigma_{\mathbf{w}}$ regardless of the choice of $\mathbf{s}$. This means hints do not carry meaningful information about $\mathbf{s}$, and therefore replacing $\mathbf{s}$ has negligible effect on $\mathcal{A}$'s view.

Similarly, from **Hybrid**₂ onward, the computation of $\mathbf{w}$ depends on the depth-specific error $\mathbf{e}$ through the added term $c \cdot \mathbf{e} + \mathbf{e}'$, which can be recovered with the help of $\mathbf{h}$. The same logic applies here, as $\mathbf{e}'$ is also sampled from a Gaussian of width $\sigma_{\mathbf{w}}$.

Thus, the difference between **Hybrid**₃ and **Hybrid**₄ is negligible, and this is already accounted for by the Hint-MLWE advantage.

Hybrid₅: The challenger now replaces the uniformly random $\mathbf{t}$ with one correctly computed from $\mathbf{s}$ instead. Similar to the argument in **Hybrid**₃, by the hardness of $\text{H-MLWE}_{k,\ell,\sigma_{\mathbf{t}},\sigma_{\mathbf{w}},Q}$, the adversary's advantage is negligible.

Now that depth n signatures are replaced with depth 1 signatures, $\mathcal{A}$ still has negligible advantage in the unlinkability game. This concludes the proof.

6 Implementation

We implement both wallet constructions in Rust, with a Python bridge to the Raccoon-G reference code [DKM⁺24] for the underlying operations. The single substantive implementation change is that Raccoon-G public keys are stored as the full, unrounded vector $\mathbf{t}$; rounding to $\hat{\mathbf{t}}$ is performed inline at the point of signing and verification. This preserves the linear structure required for non-hardened derivation while keeping the signing algorithm mathematically identical to the base Raccoon-G specification. Child key derivation, path parsing, and domain separation follow BIP32 conventions using HMAC-SHA512.

The implementation is fully deterministic and validated by a test suite covering key generation, hardened and non-hardened path derivation, signing and verification at depth, and watch-only child derivation for Raccoon-G. Each implementation targets 128 bits of security, which is consistent with BIP32. Concrete key and signature sizes for each construction are given in Table 4. Source code, parameters, and build instructions are available at <https://github.com/p-11/lattice-hd-wallets> (commit hash 461a5ed). This is a proof-of-concept: it is unaudited, unoptimized, and not intended for production use.

7 Discussion

This work demonstrates that the full functionality of BIP32 hierarchical deterministic wallets, including non-hardened public key derivation, can be recovered in the post-quantum setting with provable security under standard lattice assumptions. We now discuss the practical implications, limitations, and open questions that follow from our constructions.

Standardization vs. functionality. Our two constructions reflect a fundamental tension in the current post-quantum landscape. The ML-DSA HD Wallet (Section 4) offers the compliance advantages of a NIST-standardized primitive but is structurally limited to hardened derivation. The Raccoon-G HD Wallet (Section 5) recovers non-hardened derivation but relies on a signature scheme that is no longer on a standardization track. For deployments where regulatory or compliance requirements mandate standardized primitives, the ML-DSA construction provides a secure baseline with formal security proofs which is an improvement over existing proposals such as QIP2 [Smi], which lack any security analysis. For deployments where watch-only wallets, auditable payment systems, or scalable address generation are operational requirements, the Raccoon-G construction is, to our knowledge, the only option with concrete security guarantees.

Key and signature sizes. A practical consequence of our Raccoon-G construction is increased key and signature sizes relative to both classical BIP32 and the ML-DSA HD Wallet. Raccoon-G publishes the full, unrounded public key $\mathbf{t}$ rather than the rounded $\hat{\mathbf{t}}$ used by ML-DSA, resulting in larger public keys. Raccoon-G signatures are also substantially larger than ML-DSA signatures. Since public keys and signatures are often stored on-chain, this increase has direct implications for transaction sizes, storage costs, and network bandwidth. We note, however, that this tradeoff is intrinsic to the approach: the unrounded public key is precisely what enables the linear structure required for non-hardened derivation. Schemes that rely on bit-dropping for key compression face a similar tradeoff, as the rounding step that reduces key sizes is fundamentally incompatible with the linear operations required for public key rerandomization.

Side-channel considerations. The Raccoon framework was designed for efficient masking, and its signing procedure contains no rejection sampling loop, eliminating a known class of timing side-channels present in ML-DSA and Dilithium [DKM⁺24]. However, Raccoon-G replaces the sum-of-uniforms error distribution used in the base Raccoon submission with discrete Gaussian sampling, which is necessary for the distributional stability that enables non-hardened derivation. Constant-time implementation of discrete Gaussian sampling is a known challenge, and the side-channel properties of a concrete Raccoon-G implementation require independent analysis.

Bounded derivation depth. Unlike classical BIP32, where non-hardened derivation does not alter the distribution of derived keys regardless of depth, our

Raccoon-G construction imposes a maximum derivation depth n . Each level of non-hardened derivation adds noise to the secret key, widening its Gaussian distribution by a factor of $\sqrt{t}$ at depth t . This has two consequences: the signature verification bound $B_{2,z}$ grows with n (Lemma 1), and the Rényi divergence between depth-1 and depth- n signature distributions must remain bounded for unlinkability to hold (Corollary 2). The parameter n is fixed at key generation time and constrains the tree structure.

In practice, this limitation is less restrictive than it may appear. The dominant deployment pattern for classical HD wallets follows BIP44, which uses hardened derivation for the first three levels (purpose, coin type, account) and non-hardened derivation only for the final two (change, address index). Our construction naturally accommodates this pattern: hardened nodes (via CKDer) reset the noise, and non-hardened subtrees (via $\text{CKDer}_{\text{pub}}$) branch from these nodes. Remark 2 describes this hybrid approach, which bounds the non-hardened depth per hardened root while preserving the full tree structure.

Chain code security. Our constructions inherit a well-known property of classical BIP32. In the non-hardened setting, the offset $(\mathbf{s}', \mathbf{e}')$ used to derive a child key is computed deterministically from the parent’s public key and chain code, both of which are available to anyone performing public key derivation. If an attacker obtains a child’s secret key, they can recompute this offset and subtract it to recover the parent’s secret key, and from there derive every sibling key in the subtree. This is identical to the corresponding risk in classical BIP32, where knowledge of a child private key and the parent chain code compromises the parent private key. Hardened nodes prevent this by deriving the offset from the parent *private* key, so an attacker who compromises a child secret key but does not hold the parent private key cannot reverse the derivation.

Implications for post-quantum blockchains. The transition of blockchain systems to post-quantum cryptography is often framed as a straightforward primitive replacement: swap ECDSA for ML-DSA. Our work shows that this framing is incomplete. HD wallets are one example of an ecosystem-level extension whose security properties depend on algebraic structure that does not survive the transition to lattices without careful construction. Other extensions, multisignatures, adaptor signatures, threshold signing, stealth addresses, face analogous challenges. A complete post-quantum blockchain requires not only secure primitives but secure *compositions* of those primitives with the extended functionality that users and infrastructure depend on.

Future work. Several directions remain open. First, concrete parameter instantiations with exact key and signature sizes for each security level would enable direct comparison with deployed systems and inform protocol design decisions. Second, the Raccoon-G framework’s native support for threshold signing [DKM⁺24] suggests a natural extension to threshold HD wallets, where multiple parties collectively manage a key tree without any single party holding the master secret.

Third, the bounded derivation depth motivates investigation of noise management techniques, such as periodic key refresh or structured noise cancellation, that could extend the effective tree depth without compromising unlinkability. Finally, as the post-quantum standardization landscape evolves, revisiting our constructions with future NIST-standardized signature schemes that may offer compatible algebraic structure remains an important practical goal.

References

- ABB⁺20. Erdem Alkim, Paulo S. L. M. Barreto, Nina Bindel, Juliane Krämer, Patrick Longa, and Jefferson E. Ricardini. The lattice-based digital signature scheme qTESLA. In Mauro Conti, Jianying Zhou, Emiliano Casalicchio, and Angelo Spognardi, editors, *ACNS 2020: 18th International Conference on Applied Cryptography and Network Security, Part I*, volume 12146 of *Lecture Notes in Computer Science*, pages 441–460, Rome, Italy, October 19–22, 2020. Springer, Cham, Switzerland.
- ADE⁺20. Nabil Alkeilani Alkadri, Poulami Das, Andreas Erwig, Sebastian Faust, Juliane Krämer, Siavash Riahi, and Patrick Struck. Deterministic wallets in a quantum world. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020: 27th Conference on Computer and Communications Security*, pages 1017–1031, Virtual Event, USA, November 9–13, 2020. ACM Press.
- DEF⁺21. Poulami Das, Andreas Erwig, Sebastian Faust, Julian Loss, and Siavash Riahi. The exact security of BIP32 wallets. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021: 28th Conference on Computer and Communications Security*, pages 1020–1042, Virtual Event, Republic of Korea, November 15–19, 2021. ACM Press.
- DEMS24. Poulami Das, Andreas Erwig, Michael Meyer, and Patrick Struck. Efficient post-quantum secure deterministic threshold wallets from isogenies. In Jianying Zhou, Tony Q. S. Quek, Debin Gao, and Alvaro A. Cárdenas, editors, *ASIACCS 24: 19th ACM Symposium on Information, Computer and Communications Security*, Singapore, July 1–5, 2024. ACM Press.
- DFL19. Poulami Das, Sebastian Faust, and Julian Loss. A formal treatment of deterministic wallets. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019: 26th Conference on Computer and Communications Security*, pages 651–668, London, UK, November 11–15, 2019. ACM Press.
- DKL⁺18. Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Dilithium: A lattice-based digital signature scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2018(1):238–268, 2018.
- DKM⁺24. Rafaël Del Pino, Shuichi Katsumata, Mary Maller, Fabrice Mouhartem, Thomas Prest, and Markku-Juhani O. Saarinen. Threshold raccoon: Practical threshold signatures from standard lattice assumptions. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024, Part II*, volume 14652 of *Lecture Notes in Computer Science*, pages 219–248, Zurich, Switzerland, May 26–30, 2024. Springer, Cham, Switzerland.
- dPKPR24. Rafaël del Pino, Shuichi Katsumata, Thomas Prest, and Mélissa Rossi. Raccoon: A masking-friendly signature proven in the probing model. Cryptology ePrint Archive, Paper 2024/1291, 2024.
- DZLZ23. Chenghe Dong, Jianhong Zhang, Zongyi Lv, and Ruxuan Zhang. Lightweight hierarchical deterministic wallet supporting stealth address for iot. In *2023 IEEE 22nd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, pages 387–393, 2023.

- ER22. Andreas Erwig and Siavash Riahi. Deterministic wallets for adaptor signatures. In Vijayalakshmi Atluri, Roberto Di Pietro, Christian Damsgaard Jensen, and Weizhi Meng, editors, *ESORICS 2022: 27th European Symposium on Research in Computer Security, Part II*, volume 13555 of *Lecture Notes in Computer Science*, pages 487–506, Copenhagen, Denmark, September 26–30, 2022. Springer, Cham, Switzerland.
- KLSS23. Duhyeong Kim, Dongwon Lee, Jinyeong Seo, and Yongsoo Song. Toward practical lattice-based proof of knowledge from hint-MLWE. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part V*, volume 14085 of *Lecture Notes in Computer Science*, pages 549–580, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland.
- LS15. Adeline Langlois and Damien Stehlé. Worst-case to average-case reductions for module lattices. *Designs, Codes and Cryptography*, 75(3):565–599, 2015.
- MP13. Daniele Micciancio and Chris Peikert. Hardness of SIS and LWE with small parameters. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology – CRYPTO 2013, Part I*, volume 8042 of *Lecture Notes in Computer Science*, pages 21–39, Santa Barbara, CA, USA, August 18–22, 2013. Springer Berlin Heidelberg, Germany.
- Nak08. Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. <https://bitcoin.org/bitcoin.pdf>, 2008.
- NB⁺07. Satoshi Nakamoto, Bit Bit, et al. Bitcoin: A peer-to-peer electronic cash system. *2008*, 2007.
- NIS25. Nist releases first 3 finalized post-quantum encryption standards, Aug 2025. <https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards>.
- oST25. National Institute of Standards and Technology. Module-lattice-based digital signature standard. Technical Report Federal Information Processing Standards Publications (FIPS) 204, Change Notice 2 December 03, 2002, U.S. Department of Commerce, Washington, D.C., 2025.
- SD25. Surbhi Shaw and Ratna Dutta. Post-quantum secure compact deterministic wallets from isogeny-based signatures with rerandomized keys. *Theor. Comput. Sci.*, 1035(C), April 2025.
- Smi. Chris Smith. *Hierarchical Deterministic Wallets For Lattice Keys*. <https://github.com/Quantus-Network/improvement-proposals/blob/main/qip-0002.md>.
- SRL25. Michel Seck and Adeline Roux-Langlois. Towards post-quantum bitcoin blockchain using dilithium signature. *IACR Communications in Cryptology*, 2(3), 2025.
- Wui13. Pieter Wuille. BIP 32: Hierarchical deterministic wallets. Bitcoin Improvement Proposal, 2013.
- YL24. Xin Yin and Zhen Liu. A secure hierarchical deterministic wallet with stealth address from lattices. *Theoretical Computer Science*, 1009:114672, 2024.
- ZXE⁺23. Liyi Zhou, Xihan Xiong, Jens Ernstberger, Stefanos Chaliasos, Zhipeng Wang, Ye Wang, Kaihua Qin, Roger Wattenhofer, Dawn Song, and Arthur Gervais. SoK: Decentralized finance (DeFi) attacks. In *2023 IEEE Symposium on Security and Privacy*, pages 2444–2461, San Francisco, CA, USA, May 21–25, 2023. IEEE Computer Society Press.

Appendices

A Cryptographic Background

A.1 Digital Signatures

A digital signature scheme SIG is a tuple of algorithms $\text{SIG} = (\text{KeyGen}, \text{Sign}, \text{Verify})$ where:

- KeyGen takes parameters 1^λ as input and outputs a private signing key sk and a public verification key pk .
- Sign takes a message μ and a signing key sk outputs a signature Ψ .
- Verify takes a message μ , a signature Ψ , and a verification key pk and outputs 1 if Ψ is valid and 0 otherwise.

By assumption, KeyGen and Sign are randomized algorithms. For wallet constructions, it is important to have the possibility of deterministic equivalents of these operations. We introduce the algorithms DetKeyGen and DetSign for randomness ρ from the randomness space χ defined by the parameters 1^λ :

- DetKeyGen takes parameters 1^λ and randomness ρ as input and outputs a private signing key sk and a public verification key pk .
- DetSign takes a message μ , a signing key sk , and randomness ρ as input and outputs a signature Ψ .

Finally, for schemes with rerandomizable public keys, we add a subprotocol that changes the public key and a subprotocol to update the corresponding signing key:

- RandPK takes a public key pk , and randomness ρ and outputs a new public key pk' .
- RandSK takes a secret key sk , and randomness ρ and outputs a new secret key sk' .

The simplest property we require is *correctness* where every honestly signed message-signature pair verifies with probability 1.

For signature schemes, the main security notion that we consider is *existential unforgeability under chosen message attacks*. Informally, any attacker who does not possess sk , cannot forge a valid message-signature pair if the message has not been queried previously. More formally, any adversary in the unforgeability game in Figure 6 has negligible advantage.

The extension of this notion to the rerandomizable signature schemes is the *unforgeability under honestly rerandomized keys* [DFL19] which is a weaker notion that is valid for any rerandomization of pk ⁴. Similar to basic unforgeability,

⁴ There are even more restricted versions for both of these security notions [DEF⁺21], which allows the attacker to query an oracle only once per message (for existential unforgeability) or the oracle can return messages signed under *any* rerandomization of sk instead.

$\text{Exp}_{\text{SIG}}^{\text{UF-CMA}}(\mathcal{A})$	$\mathcal{O}^{\text{sign}}(\mu)$
$\mathcal{M} := \emptyset$	$\Psi := \text{SIG.Sign}(\text{sk}, \mu)$
$\text{pk}, \text{sk} \leftarrow \text{SIG.KeyGen}(1^\lambda)$	$\mathcal{M} := \mathcal{M} \cup \{\mu\}$
$(\Psi^*, \mu^*) \leftarrow \mathcal{A}(\mathcal{O}^{\text{sign}}, \text{pk})$	return Ψ
return $(\mu^* \notin \mathcal{M}) \wedge (\text{SIG.Verify}(\text{pk}, \Psi^*, \mu^*))$	

Fig. 6. UF-CMA experiment $\text{Exp}_{\text{SIG}}^{\text{UF-CMA}}$.

the unforgeability under honestly rerandomized key game is depicted in Figure 7.

$\text{Exp}_{\text{SIG}}^{\text{UF-CMA-HRK}}(\mathcal{A})$	$\mathcal{O}^{\text{sign}}(\mu, \rho)$
$\mathcal{M} := \emptyset; \mathcal{S} := \{\epsilon\}$	if $\rho \notin \mathcal{S}$
$\text{pk}, \text{sk} \leftarrow \text{SIG.KeyGen}(1^\lambda)$	return $\perp$
$(\Psi^*, \mu^*, \rho^*) \leftarrow \mathcal{A}(\mathcal{O}^{\text{sign}}, \mathcal{O}^{\text{rand}}, \text{pk})$	$\text{sk}' := \text{SIG.RandSK}(\text{sk}', \rho)$
$\text{pk}^* := \text{SIG.RandPK}(\text{pk}, \rho^*)$	$\Psi := \text{SIG.Sign}(\text{sk}', \mu)$
return $\mu^* \notin \mathcal{M}$	$\mathcal{M} := \mathcal{M} \cup \{\mu\}$
$\wedge \rho^* \in \mathcal{S}$	return Ψ
$\wedge \text{SIG.Verify}(\text{pk}^*, \Psi^*, \mu^*)$	
	$\mathcal{O}^{\text{rand}}$
	$\rho \leftarrow \mathcal{P}; \mathcal{S} := \mathcal{S} \cup \{\rho\}$
	return ρ

Fig. 7. UF-CMA-HRK experiment $\text{Exp}_{\text{SIG}}^{\text{UF-CMA-HRK}}$. $\mathcal{P}$ denotes the randomness domain.

In our constructions, we use two signature schemes: a variant of Dilithium [DKL⁺18] and a variant of Raccoon that uses Gaussian secrets [DKM⁺24]. For the base case scenario without public key rerandomization, we use the standardized Dilithium variant known as ML-DSA [oST25] with bounded uniform secrets. However, for the primary contribution of our HD wallet architecture, we utilize Raccoon-G, which is a Raccoon variant that uses Gaussian secrets. Unlike schemes with bounded uniform secrets, the Gaussian secrets in Raccoon-G are vital for wallet constructions because they are stable under addition. This property allows for public key rerandomization while ensuring that child public keys follow the same distribution as master public keys, maintaining unlinkability. We give this version as a reference in Appendix B.

A.2 Cyclotomic Rings, Norms, and Rounding

Consider the cyclotomic rings $\mathcal{R} = \mathbb{Z}[x]/\phi$ and $\mathcal{R}_q = \mathbb{Z}_q[x]/\phi$, where $\phi = (x^N + 1)$ for N an integer power of 2 and q a prime. Elements in these rings are polynomials with a degree of at most $d - 1$. For $\mathcal{R}_q$, we constrain the coefficients to the range $[-(q-1)/2, (q-1)/2]$. We adopt the following notation:

- **Lowercase letters** (a, b, c): Elements of $\mathbb{Z}$ or R .
- **Bold lower-case** ($\mathbf{v}$): Vectors in R^k .
- **Bold upper-case** ($\mathbf{A}$): Matrices in $R^{k \times \ell}$.

We measure the size of these elements using norms ($\|\cdot\|_1, \|\cdot\|_2$, or $\|\cdot\|_\infty$). For a matrix $\mathbf{A}$, the notation $\|\mathbf{A}\| \leq B$ implies that the norm of *every individual entry* in the matrix is bounded by B .

Bit Droppings and Embeddings We define the set $\mathcal{C}_\nu = \{c \in \mathcal{R}_q : \|c\|_\infty = 1, \|c\|_1 = \nu\} \subset \mathcal{R}_q$ as the set of polynomials with coefficients in $\{-1, 0, 1\}$ and exactly ν non-zero coefficients. We then allow $\bar{\mathcal{C}}_\nu = \{c - c' : c, c' \in \mathcal{C}_\nu, c \neq c'\}$. Let $\lfloor \cdot \rfloor : \mathbb{R} \rightarrow \mathbb{Z}$ denote the rounding-up operation. To manage precision, we define the lower κ bit dropping operation $\lfloor \cdot \rfloor_{q'} : \mathbb{Z}_q \rightarrow \mathbb{Z}_{q'}$. This map effectively cleans the data by removing the least significant digits in the manner of: $\lfloor x \rfloor_{q'} := \lfloor x/2^\kappa \rfloor + q'\mathbb{Z}$ where $q' = \lfloor q/2^\kappa \rfloor$ for $\kappa \neq 0 \in \mathbb{N}$. Like other ring operations, the bit dropping operation extends into vectors naturally by applying everything coefficient-wise. We also define the map coefficient embedding by the map $\Phi : \mathcal{R}_q \rightarrow \mathbb{Z}_q^N$ which maps elements of $\mathcal{R}_q$ to their coefficient vectors. This extends naturally to vectors and matrices of elements in $\mathcal{R}_q$.

A.3 Gaussians

For one of the underlying signature schemes, we use the standard discrete Gaussian distribution of support $\mathbb{Z}^N$, center 0, and standard deviation σ defined by its probability distribution function

$$\mathcal{D}_\sigma(z) := \frac{\rho_\sigma(z)}{\sum_{z \in \mathbb{Z}^N} \rho_\sigma(z)},$$

where $\rho := \exp\left(\frac{-\|z\|_2^2}{2\sigma^2}\right)$. Moreover, we will always use a *truncated* Gaussian distribution, from which sampling yields N -dimensional integer vectors with bounded infinity and two-norms. Specifically, sampling from this truncated Gaussian will be denoted ' $z \leftarrow \mathcal{D}_\sigma$ '⁵ and, for real numbers $t_\infty > 0$ and $t_2 > 1$, is implemented as follows:

We choose t_∞ and t_2 so that Step 2 occurs with small probability. For the properties of the randomized keys, we also rely on the following property of sum of Gaussians:

⁵ We abuse this notation in places, for example, denoting by $\mathcal{D}_\mathbf{y}$ the Gaussian from which secrets $\mathbf{s}$ and $\mathbf{e}$ are sampled to construct $\mathbf{y} := \mathbf{A}\mathbf{s} + \mathbf{e}$ as in the signing public key.

Truncated Gaussian Sampling, denoted ' $z \leftarrow \mathcal{D}$ '
1. Sample z from the regular (non-truncated) Gaussian of parameter σ and center 0. 2. if either $\ z\ _\infty \geq t_\infty \cdot \sigma$ or $\ z\ _2 \geq t_2 \cdot \sigma \cdot \sqrt{N}$ then restart 3. return z.

Fig. 8. Truncated Gaussian sampling.

Lemma 2. [MP13, Theorem 3.3] *Let σ be a parameter exceeding the smoothing parameter by a factor of at least $\sqrt{2}$ and $\mathbf{x}_i$ for $i \in [n]$ be independent samples from $\mathcal{D}_s^m$. Then the distribution of $\mathbf{x} := \sum_i \mathbf{x}_i$ is statistically close to $\mathcal{D}_{s\sqrt{n}}^m$.*

A.4 Lattice Assumptions

The security of the underlying signature scheme for wallets relies on computational problems over structured lattices. These problems can be thought of as linear algebra problems with the addition of errors. The problems we rely on are Module Short Integer Solution (MSIS), Self-Target Module Short Integer Solution (SelfTargetMSIS), Module Learning with Errors (MLWE), and Hint Module Learning with Errors (H-MLWE) over some fixed polynomial ring R_q implicitly defining the parameters q and d .

Definition 3 (MSIS [LS15]). *Let k, ℓ be positive integers and $0 < \eta \ll q$. Then, given $\mathbf{A} \leftarrow \mathcal{R}_q^{k \times \ell}$, the Module-SIS problem asks an adversary $\mathcal{A}$ to find $\mathbf{z} \in \mathcal{R}_q^\ell$ such that $\mathbf{A}\mathbf{z} = \mathbf{0}$ and $0 < \|\mathbf{z}\|_2 \leq \eta$. $\mathcal{A}$ is said to have advantage ϵ_{MSIS} in solving $\text{MSIS}_{k, \ell, \eta}$ if*

$$\Pr [0 < \|\mathbf{z}\|_2 \leq \eta \wedge \mathbf{A}\mathbf{z} = \mathbf{0} \mid \mathbf{A} \leftarrow \mathcal{R}_q^{k \times \ell}; \mathbf{z} \leftarrow \mathcal{A}(\mathbf{A})] \geq \epsilon_{\text{MSIS}}.$$

Definition 4 (MLWE [LS15]). *Let k, ℓ be positive integers, and χ be a probability distribution over $\mathcal{R}_q$. The Module-LWE problem then asks an adversary $\mathcal{A}$ to distinguish between the following two cases:*

1. $(\mathbf{A}, \mathbf{As})$ for public $\mathbf{A} \leftarrow \mathcal{R}_q^{k \times (\ell+k)}$ and secret $\mathbf{s} \leftarrow \chi^{\ell+k}$,
2. $(\mathbf{A}, \mathbf{b}) \leftarrow \mathcal{R}_q^{k \times (\ell+k)} \times \mathcal{R}_q^k$ where both are sampled uniformly.

Then $\mathcal{A}$ is said to have advantage ϵ_{MLWE} in solving $\text{MLWE}_{k, \ell, \chi}$ if

$$\left| \Pr [b = 1 \mid \mathbf{A} \leftarrow \mathcal{R}_q^{k \times (\ell+k)}; \mathbf{s} \leftarrow \chi^{\ell+k}; b \leftarrow \mathcal{A}(\mathbf{A}, \mathbf{As})] - \Pr [b = 1 \mid \mathbf{A} \leftarrow \mathcal{R}_q^{k \times (\ell+k)}; \mathbf{b} \leftarrow \mathcal{R}_q^k; b \leftarrow \mathcal{A}(\mathbf{A}, \mathbf{b})] \right| \geq \epsilon_{\text{MLWE}}.$$

When χ is $\mathcal{D}_\sigma$, we simply denote it as $\text{MLWE}_{k, \ell, \sigma}$.

Remark 3. We note the trivial hierarchy that $\text{MLWE}_{k,\ell,\sigma_0}$ is at least as hard as $\text{MLWE}_{k,\ell,\sigma_1}$ when $\sigma_0 = \sqrt{k} \cdot \sigma_1$ for any integer $k > 1$ and $\sigma_1 \geq \sqrt{2} \cdot \eta_\varepsilon(\mathcal{R})$. Given samples from the latter $(\mathbf{A}, \mathbf{b})$, an adversary can submit $(\mathbf{A}, \mathbf{b} + \mathbf{A} \cdot \delta)$ to the distinguisher for the former, where $\delta \leftarrow \mathcal{D}_{\sqrt{k-1}\sigma_0}$. By a simple corollary of Lemma 2, we have that $\delta + \mathbf{s}$ is correctly distributed if $\mathbf{s} \leftarrow \mathcal{D}_{\sigma_0}$. Since $\sum_{i=0}^{k-1} \mathcal{D}_{\sigma_0} \approx_s \mathcal{D}_{\sigma_1}$ and $\sum_{i=0}^{k-2} \mathcal{D}_{\sigma_0} \approx_s \mathcal{D}_{\sqrt{k-1}\sigma_0}$, we have $\mathcal{D}_{\sigma_0} + \mathcal{D}_{\sigma_0\sqrt{k-1}} \approx_s \mathcal{D}_{\sigma_1}$. Here, “ $\approx_s$ ” indicates that two distributions are statistically close.

Definition 5 (SelfTargetMSIS [DKL⁺18]). *Let k, ℓ be positive integers and $0 < \eta \ll q$. Let $\mathbf{H}: \mathcal{R}_q^k \times \{0, 1\}^* \rightarrow \mathcal{C}_\nu$ be a cryptographically secure hash function modeled as a random oracle. Then, given $(\mathbf{A}, \mathbf{t}) \in \mathcal{R}_q^{k \times \ell} \times \mathcal{R}_q^k$, the Self-Target MSIS problem asks an adversary $\mathcal{A}$ to find $(\mathbf{z}, \mu) \in \mathcal{R}_q^\ell \times \{0, 1\}^*$ such that $0 < \|\mathbf{z}\|_2 \leq \eta$ and $\mathbf{H}([\mathbf{A} \mid \mathbf{t}] \cdot \mathbf{r}, \mu) = c$, where $\mathbf{r} = [\mathbf{z} \ c]^\top$.*

$$\Pr \left[\begin{array}{l} \mathbf{H}([\mathbf{A} \mid \mathbf{t}] \cdot \mathbf{r}, \mu) = c \\ \wedge 0 < \|\mathbf{z}\|_2 \leq \eta \end{array} \mid \mathbf{A} \leftarrow \mathcal{R}_q^{k \times \ell}; \left(\mathbf{r} = \begin{bmatrix} \mathbf{z} \\ c \end{bmatrix}, \mu \right) \leftarrow \mathcal{A}^{\mathbf{H}}(\mathbf{A}, \mathbf{t}) \right] \geq \epsilon_{\text{STMSIS}}.$$

Definition 6 (H-MLWE [KLSS23]). *Let k, ℓ, Q be positive integers, χ_1 and χ_2 be probability distributions over $\mathcal{R}_q$, and $\mathcal{C}$ be a subset of $\mathcal{R}_q$. The Hint-MLWE problem $\text{H-MLWE}_{k,\ell,\chi_1,\chi_2,Q}$ then asks an adversary $\mathcal{A}$ to distinguish between the following two cases:*

1. $(\mathbf{A}, \mathbf{As}, (c_i, \mathbf{z}_i)_{i \in [Q]})$ for $\mathbf{A} \leftarrow \mathcal{R}_q^{k \times (\ell+k)}$,
2. $(\mathbf{A}, \mathbf{b}, (c_i, \mathbf{z}_i)_{i \in [Q]})$ for $\mathbf{A} \leftarrow \mathcal{R}_q^{k \times (\ell+k)}$, $\mathbf{b} \leftarrow \mathcal{R}_q^k$,

where $\mathbf{s} \leftarrow \chi_1^{\ell+k}$, $c_i \leftarrow \mathcal{C}$ for $i \in [Q]$, and $\mathbf{z}_i := c_i \cdot \mathbf{s} + \mathbf{y}_i$ where $\mathbf{y}_i \leftarrow \chi_2^{\ell+k}$ for $i \in [Q]$. We denote by ϵ_{HMLWE} the advantage of $\mathcal{A}$ in solving $\text{H-MLWE}_{k,\ell,\chi_1,\chi_2,Q}$.

$\mathcal{A}$ has advantage ϵ_{HMLWE} in solving $\text{H-MLWE}_{k,\ell,\chi_1,\chi_2,Q}$ if

$$\left| \Pr \left[\begin{array}{l} \mathbf{A} \leftarrow \mathcal{R}_q^{k \times (\ell+k)}; \mathbf{s} \leftarrow \chi_1^{\ell+k}; c_i \leftarrow \mathcal{C}; \\ b = 1 \mid \mathbf{y}_i \leftarrow \chi_2^{\ell+k}; \mathbf{z}_i := c_i \mathbf{s} + \mathbf{y}_i \text{ for } i \in [Q]; \\ b \leftarrow \mathcal{A}(\mathbf{A}, \mathbf{As}, (c_i, \mathbf{z}_i)_{i \in [Q]}) \end{array} \right] \right. \\ \left. - \Pr \left[\begin{array}{l} \mathbf{A} \leftarrow \mathcal{R}_q^{k \times (\ell+k)}; \mathbf{b} \leftarrow \mathcal{R}_q^k; \\ b = 1 \mid \mathbf{s} \leftarrow \chi_1^{\ell+k}; c_i \leftarrow \mathcal{C}; \mathbf{y}_i \leftarrow \chi_2^{\ell+k}; \\ \mathbf{z}_i := c_i \mathbf{s} + \mathbf{y}_i \text{ for } i \in [Q]; \\ b \leftarrow \mathcal{A}(\mathbf{A}, \mathbf{b}, (c_i, \mathbf{z}_i)_{i \in [Q]}) \end{array} \right] \right| \geq \epsilon_{\text{HMLWE}}.$$

B Raccoon-G: Raccoon with Gaussian Secrets

We give the Raccoon-G variant [DKM⁺24] here. We present Raccoon-G following the base specification, which rounds $\mathbf{t}$ to $\hat{\mathbf{t}}$ in key generation. In our wallet construction (Section 5), we publish the full unrounded $\mathbf{t}$ as the verification key to preserve linearity; $\hat{\mathbf{t}}$ is computed inline during Sign and Verify only.

KeyGen (1^λ)
$\mathbf{A} \leftarrow \mathcal{R}_q^{k \times \ell}$ $(\mathbf{s}, \mathbf{e}) \leftarrow \mathcal{D}_{\mathbf{t}}^\ell \times \mathcal{D}_{\mathbf{t}}^k$ $\hat{\mathbf{t}} := \lfloor \mathbf{A}\mathbf{s} + \mathbf{e} \rfloor_{\nu_{\mathbf{t}}} = \lfloor \mathbf{t} \rfloor_{\nu_{\mathbf{t}}}$ return $\text{vk} = (\mathbf{A}, \mathbf{t}), \text{sk} = \mathbf{s}$
Sign ($\text{vk}, \text{sk}, \mu$)
$\mathbf{r}, \mathbf{e}' \leftarrow \mathcal{D}_{\mathbf{w}}^\ell \times \mathcal{D}_{\mathbf{w}}^k$ $\mathbf{w} := \lfloor \mathbf{A}\mathbf{r} + \mathbf{e}' \rfloor_{\nu_{\mathbf{w}}}$ $c := \text{H}_c(\text{vk}, \mu, \mathbf{w})$ $\mathbf{z} := c \cdot \mathbf{s} + \mathbf{r}$ $\mathbf{y} := \lfloor \mathbf{A}\mathbf{z} - 2^{\nu_{\mathbf{t}}} \cdot c \cdot \hat{\mathbf{t}} \rfloor_{\nu_{\mathbf{w}}}$ $\mathbf{h} := \mathbf{w} - \mathbf{y}$ return $\Psi = (c, \mathbf{z}, \mathbf{h})$
Verify (vk, μ, Ψ)
$(c, \mathbf{z}, \mathbf{h}) = \Psi$ $c' := \text{H}_c(\text{vk}, \mu, \lfloor \mathbf{A}\mathbf{z} - 2^{\nu_{\mathbf{t}}} \cdot c \cdot \hat{\mathbf{t}} \rfloor_{\nu_{\mathbf{w}}} + \mathbf{h})$ if $c = c'$ and $\ (\mathbf{z}, 2^{\nu_{\mathbf{w}}} \cdot \mathbf{h})\ _2 \leq B_{2, \mathbf{z}}$ return 1 return 0

Fig. 9. Raccoon-G

C Omitted Proofs Regarding Raccoon-G

Here we give the updated proofs for Raccoon-G [DKM⁺24] based on the expansion of rerandomized keys.

C.1 Correctness

We first give the updated proof for Lemma 1.

Proof. It is clear to see $c' = c$. We need to show how randomized keys affect the ℓ_2 -norm of the signature. Overloading the notation, let $\hat{\mathbf{x}}$ denote the rounded-version of the vector by dropping $\nu_{\mathbf{x}}$ bits i.e. $\hat{\mathbf{x}} = \lfloor \mathbf{x} \rfloor_{\nu_{\mathbf{x}}}$. First, let's look at $\mathbf{y}$:

$$\begin{aligned}
 \hat{\mathbf{y}} &= \lfloor \mathbf{A}\mathbf{z} - 2^{\nu_{\mathbf{t}}} \cdot c \cdot \hat{\mathbf{t}} \rfloor_{\nu_{\mathbf{w}}} = \lfloor c \cdot \mathbf{A}\mathbf{s} + \mathbf{A}\mathbf{r} - 2^{\nu_{\mathbf{t}}} \cdot c \cdot \hat{\mathbf{t}} \rfloor_{\nu_{\mathbf{w}}} \\
 &= \lfloor \mathbf{w} + c \cdot (\mathbf{t} - 2^{\nu_{\mathbf{t}}} \cdot \hat{\mathbf{t}}) - c \cdot \mathbf{e} - \mathbf{e}' \rfloor_{\nu_{\mathbf{w}}} \\
 &= \lfloor \mathbf{w} + c \cdot \mathbf{a}_{\mathbf{t}} - \mathbf{a} \rfloor_{\nu_{\mathbf{w}}}
 \end{aligned}$$

for $\mathbf{a}, \mathbf{a}_t \in \mathcal{R}_q^k$. Substituting the above into $\mathbf{h}$ we have:

$$\begin{aligned}
\|2^{\nu_w} \cdot \mathbf{h}\|_2 &= \left\| 2^{\nu_w} \cdot \widehat{(\mathbf{w} - \mathbf{y})} \right\|_2 \\
&= \left\| 2^{\nu_w} \cdot (\widehat{\mathbf{w}} - (\mathbf{w} + c \cdot \mathbf{a}_t - \mathbf{a})) \mod q \right\|_2 \\
&\leq \left\| 2^{\nu_w} \cdot (c \cdot \mathbf{a}_t - \mathbf{a}) \mod q \right\|_2 + \sqrt{Nk} \cdot 2^{\nu_w} \\
&\leq \|c \cdot \mathbf{a}_t - \mathbf{a}\|_2 + \sqrt{Nk} \cdot 2^{\nu_w+1} \\
&\leq \|c \cdot \mathbf{a}_t\|_2 + \|\mathbf{a}\|_2 + \sqrt{Nk} \cdot 2^{\nu_w+1}
\end{aligned}$$

where

$$\|c \cdot \mathbf{a}_t\|_2 \leq \|c\|_1 \cdot \sqrt{Nk} \cdot 2^{\nu_t}$$

and

$$\begin{aligned}
\|(\mathbf{z}, \mathbf{a})\|_2 &\leq \|c \cdot (\mathbf{s}, \mathbf{e})\|_2 + \|\mathbf{r}, \mathbf{e}'\|_2 \\
&\leq e^{1/4} \cdot (\|c\|_1 \sigma_t \sqrt{n+1} + \sigma_w) \cdot \sqrt{N(k+\ell)}
\end{aligned}$$

from Lemmas 3.1(norm triangle inequality), 3.2(bit-dropping inequality), and 3.4(Euclidean norm of signatures without noise) of [DKM⁺24] and the Minkowski inequality. Note that we scale the first Gaussian width by $\sqrt{n+1}$ to accommodate key rerandomization per Lemma 2. Substituting $\|c\|_1 = \tau$ and combining the bounds we get $\|(\mathbf{z}, 2^{\nu_w} \cdot \mathbf{h})\|_2 \leq B_{2,\mathbf{z}}$. This concludes the proof. $\square$

C.2 Unforgeability

Here we follow the unforgeability proof of Raccoon-G [DKM⁺24, Appendix C.3] using the exact hybrids but update them as needed.

Hybrid₁: Exactly the same. This is the original unforgeability game which is the same regardless of the changed parameters.

Hybrid₂: Exactly the same. Here, challenger replaces outputs of $\mathbf{H}$ with programmed outputs. Since $\mathbf{w}$ is sampled randomly, while the distribution of $\mathbf{w}$ is the same, the underlying secrets have higher noise sampled from Gaussians of width $\sigma\sqrt{n}$. This means $\text{MLWE}_{k,\ell,\sigma_w\sqrt{n}}$ needs to be secure as well. However, due to the trivial hierarchy in MLWE samples (as noted in Remark 3) $\text{MLWE}_{k,\ell,\sigma_w\sqrt{n}}$ is secure as long as $\text{MLWE}_{k,\ell,\sigma_w}$ is secure. Hence, the expression does not change.

Hybrid₃: Exactly the same. Here, the challenger changes how $\mathbf{w}$ is computed based on the public key $\mathbf{t}$ instead. Since correctness is preserved as shown in the previous section, so does this hybrid equality.

Hybrid₄: Exactly the same. For the first part of the proof, we still note the trivial hierarchy in MLWE samples just like in **Hybrid₂** hence rerandomizes secrets should be secure as long as MLWE for base secrets are secure. For the second part, we note that our underlying **SelfTargetMSIS** problem is updated since $B_{2,\mathbf{z}}$ and consequently $B_{\text{ST-MSIS}} = B_{2,\mathbf{z}} + \sqrt{\tau} + (\tau \cdot 2^{\nu_t} + 2^{(\nu_w+1)}) \cdot \sqrt{Nk}$ (as in [DKM⁺24, Appendix C.4.] is updated.

Table 2. Parameters of the signature scheme

Parameter	Description	Constraints (if any)
(t, n)	Current iteration of key randomization and total # of allowed randomizations	$t \leq n$
N	Dimension of polynomial ring	$\text{poly}(\lambda)$
(k, ℓ)	Dimensions of public signing matrix $\mathbf{A}$	Set so that MLWE and MSIS hard
Q	max # of signatures issued per base signing key	$Q \approx 2^{60}$
$(\nu_{\mathbf{t}}, \nu_{\mathbf{w}})$	# of bits dropped for verification key $\mathbf{t}$ and the commitment $\mathbf{w}$	$O(\log \lambda), \nu_{\mathbf{w}} \geq 4$
τ	ℓ_1 norm bound for the signature challenge c for challenge space $\mathcal{C}$	$ \mathcal{C} \geq 2^\lambda$
$B_{\text{H-MLWE}}$	Bound required for the H-MLWE to MLWE reduction	$B_{\text{H-MLWE}} = Q \cdot \tau \cdot \left(1 + \frac{N(\lambda+1+2\log(N))}{\sqrt{Q}}\right)$
$\sigma_{\mathbf{t}}$	Gaussian width of user signing key secrets in $\mathbf{t}$	$\sigma_{\mathbf{t}} \geq 2\sqrt{\ell + k} \cdot \log N$
$\sigma_{\mathbf{w}}$	Gaussian width of the signature randomness in $\mathbf{w}$	$\sigma_{\mathbf{w}} \geq 2\sqrt{B_{\text{H-MLWE}} \cdot (\ell + k)} \cdot \log N,$
σ	Gaussian width for the H-MLWE to MLWE reduction	$\sigma \geq \sqrt{\ell + k} \cdot \omega(\sqrt{\log N})$ $\frac{1}{\sigma^2} = 2 \cdot \left(\frac{1}{\sigma_{\mathbf{t}}^2} + \frac{B_{\text{H-MLWE}}}{\sigma_{\mathbf{w}}^2}\right)$
$B_{2,\mathbf{z}}$	Bound for the signature $\mathbf{z}$	$B_{2,\mathbf{z}} = e^{1/4} \cdot (\sqrt{n+1}\tau\sigma_{\mathbf{t}} + \sigma_{\mathbf{w}})\sqrt{N(k+\ell)} + (\tau \cdot 2^{\nu_{\mathbf{t}}} + 2^{(\nu_{\mathbf{w}}+1)}) \cdot \sqrt{Nk}$
$B_{\text{ST-MSIS}}$	Bound required for the ST-MSIS security	$B_{\text{ST-MSIS}} = B_{2,\mathbf{z}} + \sqrt{\tau} + (\tau \cdot 2^{\nu_{\mathbf{t}}} + 2^{(\nu_{\mathbf{w}}+1)}) \cdot \sqrt{Nk}$
B_{MSIS}	Bound required for the ST-MSIS to MSIS reduction	$B_{\text{MSIS}} = 2B_{\text{ST-MSIS}}$
q	Signature modulus	$q > B_{\text{MSIS}}\sqrt{Nk} \cdot \omega(\log(Nk))$

Table 3. Comparative Analysis of HD Wallet Constructions

Feature	BIP32 (Classical)	PQ-HD (ML-DSA)	PQ-HD (Raccoon-G)
Security Basis	ECDSA (DL)	Lattice (M-LWE/SIS)	Lattice (M-LWE/SIS)
Post-Quantum	No	Yes	Yes
Non-Hardened Derivation	Supported	Not supported	Supported
Public Key Derivation	Supported	Not supported	Supported
Standardization	Legacy Industry Std	NIST Standardized	Research Variant
Hardware Implementation	Straightforward	Straightforward	Challenging
Main Pro	Universal adoption	NIST compliance	Secure non-hardened PK derivation
Main Con	Quantum vulnerable	No PK derivation	Larger signature sizes

Table 4. Key and signature sizes (bytes) across HD wallet constructions.

	BIP32 (Classical)	PQ-HD (ML-DSA)	PQ-HD (Raccoon-G)
Private key	32	2 560	16 128
Public key	33	1 312	16 144
Signature	64	2 420	20 768